

Data: Exploration and Preparation — Key Concepts

Lesson 2 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

2 October 2026 · a one-page index

Contents

Looking before touching	1
Missing values	1
Outliers	2
Scaling	2
Categorical encoding	2
Building features	3
The rule that ties it together	3

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

Looking before touching

Exploratory analysis. Reading size, types, target balance, gaps and shapes before transforming anything. It fits nothing and learns nothing, which is why it is safe to do on the whole dataset. → § 2

Pearson correlation. How much knowing that a row is above average on one column tells you it is above average on another. Unit-free, symmetric, and blind to any relationship that is not a straight line. → § 2

Redundancy between features. Two columns correlated near 1 are nearly one column entered twice: the model gets two coefficients and almost nothing to decide between them, so they swing between fits. → § 2

Missing values

Missing completely at random (MCAR). The chance a value is absent depends on nothing — not on what you observe, not on the missing value itself. → § 3.1

Missing at random (MAR). The chance depends on columns you *do* observe, so those columns can stand in for it. → § 3.1

Missing not at random (MNAR). The chance depends on the missing value itself. No imputation recovers it, and the absence is itself evidence. Diagnosed from how the data was collected,

never from the data. → § 3.1

Imputation. Filling a gap with a statistic — a mean, a median, the most frequent category, or a value predicted from other columns. → § 3.3

Attenuation. What mean imputation costs: it leaves the column's own mean unbiased while shrinking every correlation it has with anything else, by a factor of $\sqrt{1-p}$. → § 3.2

Missingness indicator. An ordinary binary column recording *whether* a value was missing, which keeps that fact available after the gap is filled. Earns its place only when no other column already carries it. → § 3.3

Outliers

Outlier. A value far from the rest of its column — far enough to look as though it came from a different process. The definition says nothing about the value being wrong. → § 4

z-score rule. Flag values more than k standard deviations from the mean. Principled under a bell curve, and computed from ingredients the outliers themselves corrupt. → § 4.1

Interquartile range (IQR), and Tukey's rule. Flag values beyond the quartiles by a multiple of their spread. Robust, because a handful of extremes cannot move a quartile. → § 4.1

Domain rule. A check of validity rather than of unusualness — a tenure cannot be negative. Catches mistakes no distributional rule can, and vice versa. → § 4.2

Winsorising. Replacing a flagged value with the fence rather than deleting the row, so every other column of that row survives. → § 4.2

Scaling

Standardisation. Centre each feature at zero and divide by its spread, so every column is measured in its own standard deviations. → § 5.1

Min-max scaling. Map each feature into $[0, 1]$. More sensitive to a single extreme value, which can compress every ordinary value into a sliver. → § 5.1

Why distance-based methods need it. They add up per-feature squared differences, so a numerically larger column dominates the sum regardless of how informative it is. → § 5.1

Condition number. How stretched the cost surface is — the ratio of the largest feature variance to the smallest. It is what sets how many gradient descent steps you need. → § 5.2

Learning rate. The single step size gradient descent applies to every direction at once, which is why the worst-scaled feature sets the pace for all of them. The algebra behind both consequences is Lesson 3's, once descent itself has been built. → § 5.2

Categorical encoding

Levels. The distinct values a categorical column can take. → § 6

One-hot encoding. One binary column per level, asserting only that the levels are different. → § 6.1

Dummy variable trap. With an intercept, all k dummies sum to the intercept column, so the design matrix loses rank and the fit stops being unique. Dropping one level repairs it. → § 6.1

Reference category. The level that is dropped and absorbed into the intercept. Nothing is lost: it is “all the other dummies are zero”. → § 6.1

Ordinal encoding. Mapping levels to integers. Appropriate only when the categories are genuinely ordered *and* equally spaced — the second condition is the one people skip. → § 6.2

Target encoding. Replacing a level with a statistic of the target over the rows sharing it. Compresses many levels into one column, and is the most dangerous encoding in the lesson. → § 6.2

Curse of dimensionality. Every column you add is a question to be answered from the same rows; add enough and the evidence per question runs out. → § 6.3

High cardinality. A column with very many levels. One-hot encoding one of them adds a column per level, which is how you walk into the curse fastest. → § 6.3

Building features

Feature engineering. Computing a new column so a model can express something it could not — a ratio, an interaction, a binned version of a continuous variable. A hypothesis about the domain, not a guaranteed improvement. → § 7

Interaction term. The product of two columns, for when the combination matters beyond either alone. → § 7

Binning. Turning a continuous column into ordered categories, which lets a linear model follow a non-monotonic relationship piecewise. → § 7

The rule that ties it together

Pipeline. An object that fits every preprocessing step inside the same call that fits the model, so no step can see held-out rows. → § 8.2

ColumnTransformer. Applies a different sub-pipeline to each named group of columns and concatenates the results. → § 8.2

Stratification. Splitting so that each class keeps the proportion it has in the whole dataset, so a rare class cannot land unevenly by chance. → § 8.2

Data leakage in preprocessing. Any step that learns from data — a mean, a set of neighbours, a per-category average — estimated using rows that later serve as test data. → § 9.1

Cross-fitting. Computing each training row’s encoded value from the *other* folds, so a row’s own label never votes on its own feature. The repair for target encoding. → § 9.2

The one test. *Does fitting this step compute anything from the rows it is given?* If yes, it belongs inside the pipeline. → § 9.3